

Ad Click Prediction Case Study

Executive Summary

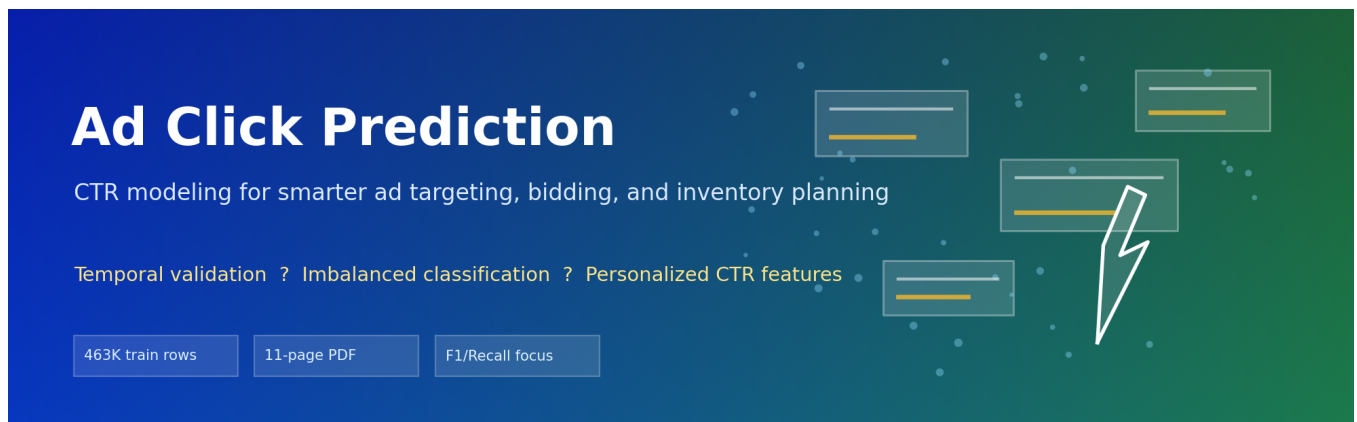
This executable notebook-style case study predicts rare ad clicks using leakage-safe temporal features, class imbalance handling, threshold tuning, ranking/lift evaluation, interpretability, and business recommendations. The format follows the earlier campaign-sales case study: business context first, then step-by-step Python implementation with selected outputs and visual evidence.

Business Context

Digital advertisers need to rank impressions by click propensity so budget is focused on users more likely to engage, while reducing wasted impressions and improving user experience.

Case Study Objectives

Inspect and clean data; engineer temporal, contextual and historical CTR features without leakage; compare baseline, Logistic Regression, Random Forest and XGBoost; evaluate PR-AUC, F1, lift, calibration, cold start and SMOTENC trade-offs; answer all seven business questions; generate final test predictions.



Step 1. Import Libraries and Configuration

This section imports all packages and defines reproducible project settings.

Step 1. Import Libraries and Configuration - Python Implementation

```
# Ad Click Prediction - corrected leakage-safe case study
import warnings
warnings.filterwarnings('ignore')

from pathlib import Path
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from IPython.display import Markdown, display, Image
```

```

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    roc_auc_score, average_precision_score, f1_score, precision_score, recall_score,
    confusion_matrix, classification_report, precision_recall_curve, roc_curve
)
from sklearn.calibration import calibration_curve
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from xgboost import XGBClassifier
from statsmodels.stats.proportion import proportions_ztest, proportion_confint
from imblearn.over_sampling import SMOTENC
import time

RANDOM_STATE = 42

```

Step 2. Load Dataset and Inspect Schema

This section loads the train/test CSV files and displays core data-quality checks.

Step 2. Load Dataset and Inspect Schema - Python Implementation

```

DATA_DIR = Path('assets/data')
IMAGE_DIR = Path('assets/images')
OUTPUT_DIR = Path('outputs')
REPORT_DIR = Path('reports')
for d in [IMAGE_DIR, OUTPUT_DIR, REPORT_DIR]:
    d.mkdir(parents=True, exist_ok=True)

TRAIN_PATH = DATA_DIR / 'Ad_click_prediction_train (1).csv'
TEST_PATH = DATA_DIR / 'Ad_Click_prediciton_test.csv'

train = pd.read_csv(TRAIN_PATH)
test = pd.read_csv(TEST_PATH)
train['DateTime'] = pd.to_datetime(train['DateTime'])
test['DateTime'] = pd.to_datetime(test['DateTime'])
print('Train:', train.shape, 'Test:', test.shape)
print('Click rate:', round(train['is_click'].mean() * 100, 2), '%')

```

Key Output

```

Train: (463291, 15) Test: (128858, 14)
Click rate: 6.76 %

```

Step 3. Feature Engineering Without Target Leakage

Temporal, behavioral and historical CTR features are created using only available past information.

Step 3. Feature Engineering Without Target Leakage - Python Implementation

```

def add_time_features(df):
    out = df.copy()
    out['hour'] = out['DateTime'].dt.hour
    out['day'] = out['DateTime'].dt.day
    out['dayofweek'] = out['DateTime'].dt.dayofweek
    out['is_weekend'] = (out['dayofweek'] >= 5).astype(int)
    out['is_night'] = out['hour'].between(0, 5).astype(int)
    out['is_business_hour'] = out['hour'].between(9, 18).astype(int)
    out['product_category_2_missing'] = out['product_category_2'].isna().astype(int)
    return out

def add_interaction_keys(df):
    out = df.copy()
    out['user_product'] = out['user_id'].astype(str) + '_' + out['product'].astype(str)
    out['campaign_product'] = out['campaign_id'].astype(str) + '_' + out['product'].astype(str)
    out['webpage_product'] = out['webpage_id'].astype(str) + '_' + out['product'].astype(str)
    return out

def add_cumulative_history_features(df, cols, target='is_click', smoothing=30, initial_prior=0.05):

```

```

"""Leakage-safe historical CTR/counts for training rows.
Each row only sees impressions/clicks that occurred before itself.
The smoothing prior is also expanding, so future clicks are not used.
"""
out = df.sort_values(['DateTime', 'session_id']).copy()
global_previous_count = np.arange(len(out), dtype=float)
global_previous_clicks = out[target].cumsum().shift(fill_value=0).astype(float)
expanding_prior = (global_previous_clicks + smoothing * initial_prior) / (global_previous_count + smoothing)
for col in cols:
    previous_count = out.groupby(col).cumcount()
    previous_clicks = out.groupby(col)[target].cumsum() - out[target]
    out[f'{col}_hist_impressions'] = previous_count.astype(float)
    out[f'{col}_hist_clicks'] = previous_clicks.astype(float)
    out[f'{col}_hist_ctr'] = (previous_clicks + smoothing * expanding_prior) / (previous_count + smoothing)
final_prior = float((out[target].sum() + smoothing * initial_prior) / (len(out) + smoothing))
return out.sort_index(), final_prior

def build_history_maps(df, cols, target='is_click', smoothing=30):
    prior = df[target].mean()
    maps = {}
    for col in cols:
        stats = df.groupby(col)[target].agg(['sum', 'count'])
        stats['hist_ctr'] = (stats['sum'] + smoothing * prior) / (stats['count'] + smoothing)
        maps[col] = {
            'ctr': stats['hist_ctr'].to_dict(),
            'clicks': stats['sum'].to_dict(),
            'impressions': stats['count'].to_dict(),
        }
    return maps, prior

def apply_history_maps(df, maps, prior, cols):
    out = df.copy()
    for col in cols:
        out[f'{col}_hist_impressions'] = out[col].map(maps[col]['impressions']).fillna(0).astype(float)
        out[f'{col}_hist_clicks'] = out[col].map(maps[col]['clicks']).fillna(0).astype(float)
        out[f'{col}_hist_ctr'] = out[col].map(maps[col]['ctr']).fillna(prior).astype(float)
    return out

train_fe = add_interaction_keys(add_time_features(train))
test_fe = add_interaction_keys(add_time_features(test))
holdout_cutoff = train_fe['DateTime'].max().normalize()
threshold_cutoff = holdout_cutoff - pd.Timedelta(days=1)
train_raw = train_fe[train_fe['DateTime'] < threshold_cutoff].copy()
threshold_raw = train_fe[(train_fe['DateTime'] >= threshold_cutoff) & (train_fe['DateTime'] <
    holdout_cutoff)].copy()
holdout_raw = train_fe[train_fe['DateTime'] >= holdout_cutoff].copy()
print('Training period ends before:', threshold_cutoff.date())
print('Threshold-tuning date:', threshold_cutoff.date())
print('Final holdout date:', holdout_cutoff.date())
print('Train:', train_raw.shape, 'Threshold:', threshold_raw.shape, 'Holdout:', holdout_raw.shape, 'Holdout click
    rate:', round(holdout_raw.is_click.mean(), 4))
print('Random stratified split is not primary because future ad-serving behavior should not leak backward into
    training.')

history_cols = [
    'product', 'campaign_id', 'webpage_id', 'product_category_1', 'product_category_2',
    'user_group_id', 'age_level', 'city_development_index', 'user_product',
    'campaign_product', 'webpage_product'
]
interaction_cols = ['user_product', 'campaign_product', 'webpage_product']

train_hist, train_prior = add_cumulative_history_features(train_raw, history_cols)
train_maps, _ = build_history_maps(train_raw, history_cols)
threshold_hist = apply_history_maps(threshold_raw, train_maps, train_prior, history_cols)
pre_holdout_history = pd.concat([train_raw, threshold_raw], axis=0)
holdout_maps, holdout_prior = build_history_maps(pre_holdout_history, history_cols)
holdout_hist = apply_history_maps(holdout_raw, holdout_maps, holdout_prior, history_cols)

# Keep model iteration practical while preserving untouched threshold and holdout periods.
model_train_idx, _ = train_test_split(
    np.arange(len(train_hist)),
    train_size=min(150000, len(train_hist)),
    stratify=train_hist['is_click'],
    random_state=RANDOM_STATE,
)
model_train = train_hist.iloc[model_train_idx].copy()
print('Model training sample:', model_train.shape, 'Threshold tuning:', threshold_hist.shape, 'Final holdout:',
    holdout_hist.shape)

baseline_features = [
    'product', 'gender', 'campaign_id', 'webpage_id', 'product_category_1', 'product_category_2',
    'user_group_id', 'age_level', 'user_depth', 'city_development_index', 'var_1',
    'hour', 'dayofweek', 'is_weekend', 'is_night', 'is_business_hour', 'product_category_2_missing'
]
interaction_count_features = [f'{c}_hist_impressions' for c in interaction_cols] + [f'{c}_hist_clicks' for c in
    interaction_cols]
historical_ctr_features = [f'{c}_hist_ctr' for c in history_cols]
all_numeric_features = interaction_count_features + historical_ctr_features
full_features = baseline_features + all_numeric_features
cat_features = baseline_features

```

```

def make_preprocess(numeric_features):
    return ColumnTransformer([
        ('cat', Pipeline([
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('onehot', OneHotEncoder(handle_unknown='ignore', min_frequency=20)),
        ]), cat_features),
        ('num', Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', StandardScaler()),
        ]), numeric_features),
    ])

def evaluate_probs(y_true, probs, name, threshold=0.5):
    preds = (probs >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, preds).ravel()
    return {
        'model': name, 'threshold': float(threshold),
        'roc_auc': roc_auc_score(y_true, probs),
        'pr_auc': average_precision_score(y_true, probs),
        'f1': f1_score(y_true, preds, zero_division=0),
        'precision': precision_score(y_true, preds, zero_division=0),
        'recall': recall_score(y_true, preds, zero_division=0),
        'tn': int(tn), 'fp': int(fp), 'fn': int(fn), 'tp': int(tp)
    }

def best_threshold_by_f1(y_true, probs):
    thresholds = np.unique(np.round(np.linspace(0.01, 0.99, 199), 3))
    f1s = [f1_score(y_true, probs >= t, zero_division=0) for t in thresholds]
    return float(thresholds[int(np.argmax(f1s))])

def fit_eval_model(name, estimator, features, numeric_features):
    pipe = Pipeline([('prep', make_preprocess(numeric_features)), ('model', estimator)])
    pipe.fit(model_train[features], model_train['is_click'])
    threshold_probs = pipe.predict_proba(threshold_hist[features])[:, 1]
    threshold = best_threshold_by_f1(threshold_hist['is_click'], threshold_probs)
    holdout_probs = pipe.predict_proba(holdout_hist[features])[:, 1]
    return pipe, threshold_probs, holdout_probs, [evaluate_probs(holdout_hist['is_click'], holdout_probs, name,
    0.5), evaluate_probs(holdout_hist['is_click'], holdout_probs, name + ' - tuned threshold', threshold)]

```

Key Output

```

Training period ends before: 2017-07-06
Threshold-tuning date: 2017-07-06
Final holdout date: 2017-07-07
Train: (314299, 25) Threshold: (77526, 25) Holdout: (71466, 25) Holdout click rate: 0.0616
Random stratified split is not primary because future ad-serving behavior should not leak backward into training.
Model training sample: (150000, 58) Threshold tuning: (77526, 58) Final holdout: (71466, 58)

```

Step 4. Baselines and Feature Ablation

Baselines and ablation runs quantify the incremental value of engineered features.

Step 4. Baselines and Feature Ablation - Python Implementation

```

# Proper baselines
baseline_rows = []
y_valid = holdout_hist['is_click']
global_ctr = model_train['is_click'].mean()
baseline_rows.append(evaluate_probs(y_valid, np.full(len(y_valid), global_ctr), 'Global CTR probability baseline',
    global_ctr))
baseline_rows.append({
    'model': 'Always non-click baseline', 'threshold': 0.5, 'roc_auc': 0.5,
    'pr_auc': y_valid.mean(), 'f1': 0.0, 'precision': 0.0, 'recall': 0.0,
    'tn': int((y_valid == 0).sum()), 'fp': 0, 'fn': int((y_valid == 1).sum()), 'tp': 0
})

# Feature ablation using the same balanced logistic estimator for clean comparison.
ablation_specs = [
    ('Baseline raw + temporal', baseline_features, []),
    ('+ Interaction history counts', baseline_features + interaction_count_features, interaction_count_features),
    ('+ Leakage-safe historical CTR', full_features, all_numeric_features),
]
ablation_rows = []
for label, feats, nums in ablation_specs:
    estimator = LogisticRegression(max_iter=500, class_weight='balanced', n_jobs=-1, random_state=RANDOM_STATE)
    _, _, probs, rows = fit_eval_model(label, estimator, feats, nums)
    tuned = [r for r in rows if r['model'].endswith('tuned threshold')][0]
    ablation_rows.append(tuned)
ablation = pd.DataFrame(ablation_rows)
display(pd.DataFrame(baseline_rows))
display(ablation)

```

Key Output

model	threshold	roc auc	pr auc	f1	precision	recall	tn	fp	fn	tp
Global CTR probability baseline	0.0704	0.5000	0.0616	0.1160	0.0616	1.0000	0	67065	0	4401
Always non-click baseline	0.5000	0.5000	0.0616	0.0000	0.0000	0.0000	67065	0	4401	0

model	threshold	roc auc	pr auc	f1	precision	recall	tn	fp	fn	tp
Baseline raw + temporal - tuned threshold	0.4900	0.5297	0.0676	0.1189	0.0664	0.5651	32116	34949	1914	2487
+ Interaction history counts - tuned threshold	0.5200	0.5767	0.0765	0.1327	0.0788	0.4222	45331	21734	2543	1858
+ Leakage-safe historical CTR - tuned threshold	0.4650	0.5937	0.0815	0.1371	0.0814	0.4363	45387	21678	2481	1920

Step 5. Model Training and Comparison

Logistic Regression, Random Forest and XGBoost are trained and evaluated with consistent metrics.

Step 5. Model Training and Comparison - Python Implementation

```
models = {
    'Logistic Regression - balanced': LogisticRegression(max_iter=500, class_weight='balanced', n_jobs=-1,
        random_state=RANDOM_STATE),
    'Random Forest - balanced': RandomForestClassifier(n_estimators=80, max_depth=10, min_samples_leaf=50,
        class_weight='balanced_subsample', n_jobs=-1, random_state=RANDOM_STATE),
    'XGBoost - weighted': XGBClassifier(n_estimators=120, max_depth=4, learning_rate=0.08, subsample=0.85,
        colsample_bytree=0.85, eval_metric='logloss',
        scale_pos_weight=(model_train['is_click'].eq(0).sum()/model_train['is_click'].eq(1).sum()),
        random_state=RANDOM_STATE, n_jobs=-1, tree_method='hist')
}
results = baseline_rows.copy()
fitted = {}
threshold_probs_by_model = {}
probs_by_model = {}
for name, estimator in models.items():
    pipe, threshold_probs, probs, rows = fit_eval_model(name, estimator, full_features, all_numeric_features)
    fitted[name] = pipe
    threshold_probs_by_model[name] = threshold_probs
    probs_by_model[name] = probs
    results.extend(rows)
metrics = pd.DataFrame(results).sort_values(['f1', 'roc_auc'], ascending=False)
display(metrics)

best_tuned = metrics[metrics['model'].str.endswith(' - tuned threshold')].iloc[0]
best_name = best_tuned['model'].replace(' - tuned threshold', '')
best_threshold = float(best_tuned['threshold'])
best_pipe = fitted[best_name]
best_threshold_probs = threshold_probs_by_model[best_name]
best_probs = probs_by_model[best_name]
print('Selected:', best_name, 'threshold:', round(best_threshold, 3))
print(classification_report(y_valid, best_probs >= best_threshold, zero_division=0))
print('Selection rationale: highest F1 among evaluated tuned-threshold classifiers; PR-AUC and ROC-AUC leaders are
    reported separately.')
```

Key Output

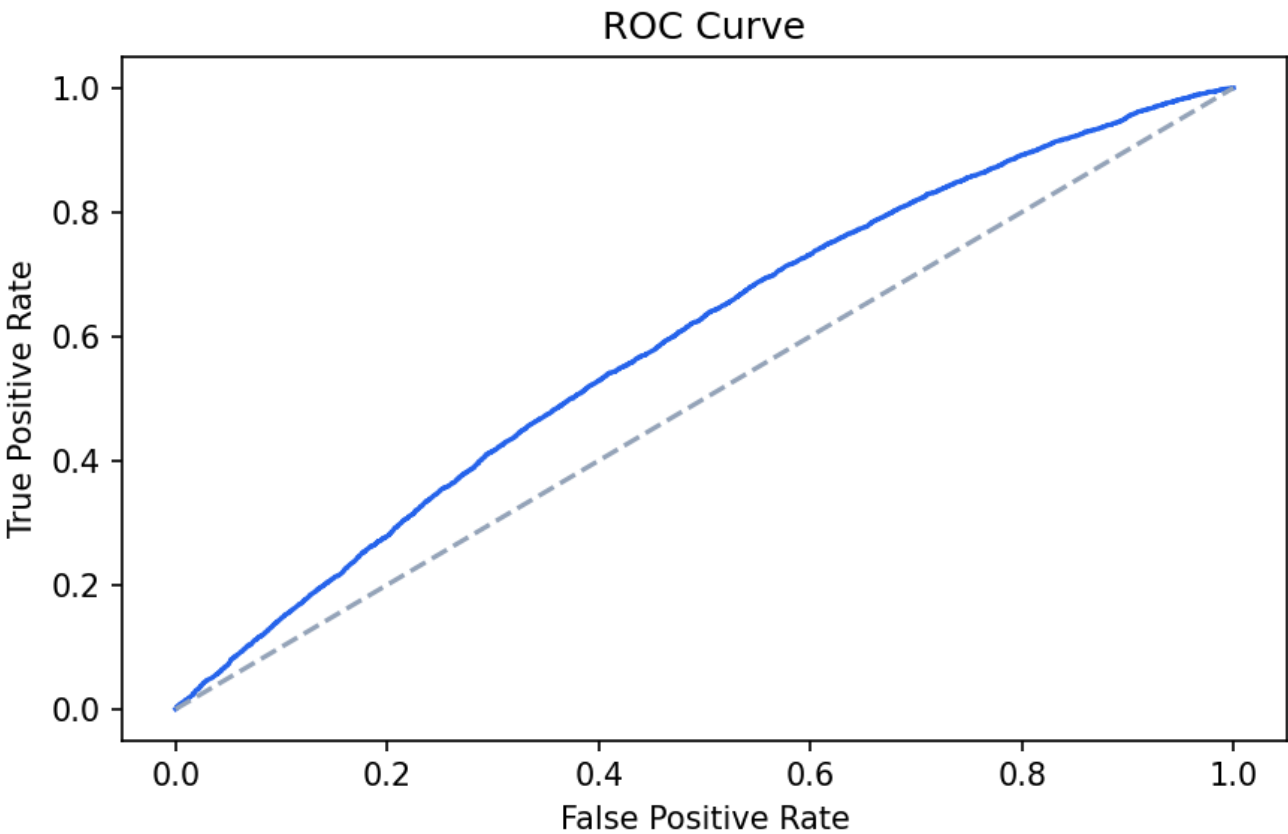
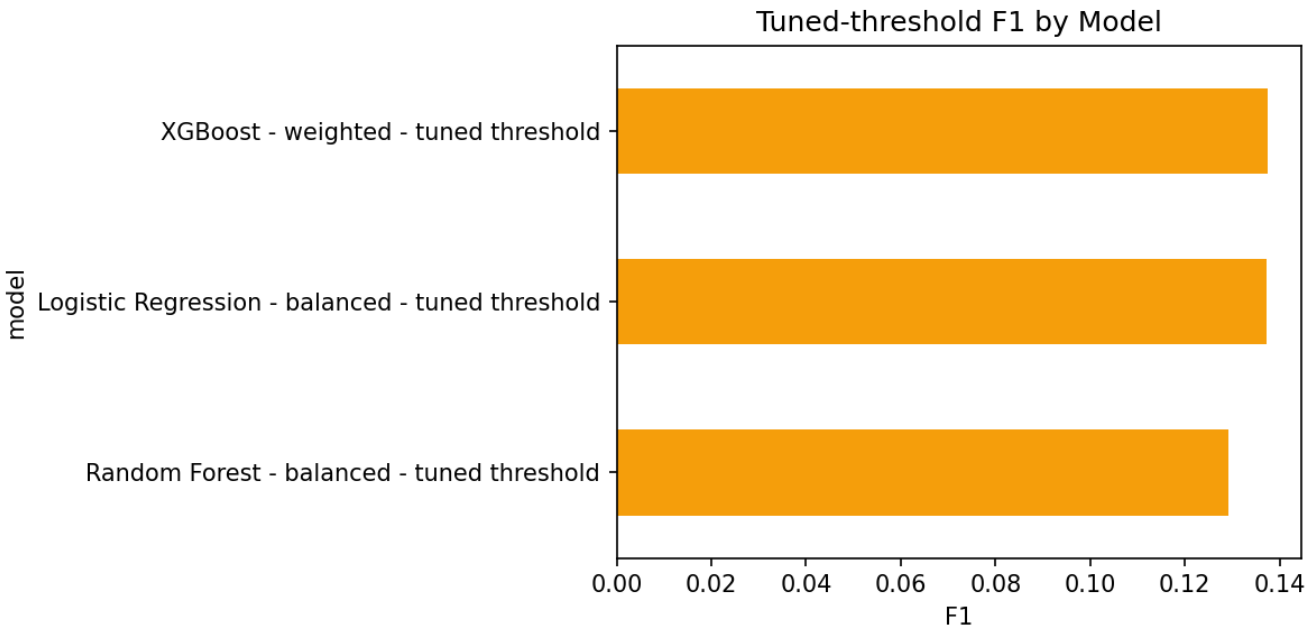
model	threshold	roc auc	pr auc	f1	precision	recall	tn	fp	fn	tp
XGBoost - weighted - tuned threshold	0.4950	0.5906	0.0795	0.1376	0.0833	0.3963	47861	19204	2657	1744
Logistic Regression - balanced - tuned threshold	0.4650	0.5937	0.0815	0.1371	0.0814	0.4363	45387	21678	2481	1920
XGBoost - weighted	0.5000	0.5906	0.0795	0.1367	0.0835	0.3761	48909	18156	2746	1655
Logistic Regression - balanced	0.5000	0.5937	0.0815	0.1347	0.0892	0.2749	54715	12350	3191	1210
Random Forest - balanced - tuned threshold	0.4650	0.5873	0.0790	0.1292	0.0860	0.2597	54919	12146	3258	1143
Global CTR probability baseline	0.0704	0.5000	0.0616	0.1160	0.0616	1.0000	0	67065	0	4401
Random Forest - balanced	0.5000	0.5873	0.0790	0.0930	0.0869	0.1000	62441	4624	3961	440

model	threshold	roc auc	pr auc	f1	precision	recall	tn	fp	fn	tp
Always non-click baseline	0.5000	0.5000	0.0616	0.0000	0.0000	0.0000	67065	0	4401	0

Selected: XGBoost - weighted threshold: 0.495

	precision	recall	f1-score	support
0	0.95	0.71	0.81	67065
1	0.08	0.40	0.14	4401
accuracy			0.69	71466
macro avg	0.52	0.55	0.48	71466
weighted avg	0.89	0.69	0.77	71466

Selection rationale: highest F1 among evaluated tuned-threshold classifiers; PR-AUC and ROC-AUC leaders are reported separately



Step 6. SMOTENC vs Class Weighting Experiment

A controlled imbalance-handling experiment compares class weighting and SMOTENC before one-hot encoding.

Step 6. SMOTENC vs Class Weighting Experiment - Python Implementation

```
# Controlled SMOTENC experiment for the explicit business question.
# SMOTENC is applied before one-hot encoding, so categorical variables are not interpolated.
smote_numeric_features = all_numeric_features
smote_features = cat_features + smote_numeric_features
smote_train_idx, _ = train_test_split(
    np.arange(len(model_train)),
    train_size=min(50000, len(model_train)),
    stratify=model_train['is_click'],
    random_state=RANDOM_STATE,
)
smote_train = model_train.iloc[smote_train_idx].copy()
smote_y = smote_train['is_click']
smote_threshold_y = threshold_hist['is_click']
smote_holdout_y = holdout_hist['is_click']

smote_cat_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('ordinal', OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)),
])
smote_num_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
])
smote_preprocess = ColumnTransformer([
    ('cat', smote_cat_transformer, cat_features),
    ('num', smote_num_transformer, smote_numeric_features),
])

start = time.perf_counter()
X_smote_base = smote_preprocess.fit_transform(smote_train[smote_features], smote_y)
X_smote_threshold = smote_preprocess.transform(threshold_hist[smote_features])
X_smote_holdout = smote_preprocess.transform(holdout_hist[smote_features])
cat_indices = list(range(len(cat_features)))

weighted_lr = LogisticRegression(max_iter=500, class_weight='balanced', n_jobs=-1, random_state=RANDOM_STATE)
weighted_lr.fit(X_smote_base, smote_y)
weighted_threshold_scores = weighted_lr.predict_proba(X_smote_threshold)[: , 1]
weighted_threshold = best_threshold_by_f1(smote_threshold_y, weighted_threshold_scores)
weighted_holdout_scores = weighted_lr.predict_proba(X_smote_holdout)[: , 1]
weighted_time = time.perf_counter() - start
weighted_eval = evaluate_probs(smote_holdout_y, weighted_holdout_scores, 'Class weighting sample experiment',
                              weighted_threshold)

start = time.perf_counter()
minority_count = int(smote_y.sum())
majority_count = int((smote_y == 0).sum())
target_minority = min(majority_count, minority_count * 3)
smotenc = SMOTENC(categorical_features=cat_indices, sampling_strategy={1: target_minority},
                  random_state=RANDOM_STATE, k_neighbors=5)
X_smote_resampled, y_smote_resampled = smotenc.fit_resample(X_smote_base, smote_y)
smotenc_lr = LogisticRegression(max_iter=500, n_jobs=-1, random_state=RANDOM_STATE)
smotenc_lr.fit(X_smote_resampled, y_smote_resampled)
smotenc_threshold_scores = smotenc_lr.predict_proba(X_smote_threshold)[: , 1]
smotenc_threshold = best_threshold_by_f1(smote_threshold_y, smotenc_threshold_scores)
smotenc_holdout_scores = smotenc_lr.predict_proba(X_smote_holdout)[: , 1]
smotenc_time = time.perf_counter() - start
smotenc_eval = evaluate_probs(smote_holdout_y, smotenc_holdout_scores, 'SMOTENC sample experiment',
                              smotenc_threshold)

smotenc_comparison = pd.DataFrame([
    {
        'strategy': 'Class weighting',
        'training_rows_after_sampling': len(smote_train),
        'precision': weighted_eval['precision'],
        'recall': weighted_eval['recall'],
        'f1': weighted_eval['f1'],
        'pr_auc': weighted_eval['pr_auc'],
        'false_negatives': weighted_eval['fn'],
        'training_time_sec': weighted_time,
        'threshold': weighted_threshold,
    },
    {
        'strategy': 'SMOTENC',
        'training_rows_after_sampling': len(y_smote_resampled),
        'precision': smotenc_eval['precision'],
        'recall': smotenc_eval['recall'],
```

```

        'f1': smotenc_eval['f1'],
        'pr_auc': smotenc_eval['pr_auc'],
        'false_negatives': smotenc_eval['fn'],
        'training_time_sec': smotenc_time,

        'threshold': smotenc_threshold,
    }
})
base_fn = smotenc_comparison.loc[smotenc_comparison['strategy'].eq('Class weighting'), 'false_negatives'].iloc[0]
smote_fn = smotenc_comparison.loc[smotenc_comparison['strategy'].eq('SMOTENC'), 'false_negatives'].iloc[0]
base_rows = smotenc_comparison.loc[smotenc_comparison['strategy'].eq('Class weighting'),
    'training_rows_after_sampling'].iloc[0]
smote_rows = smotenc_comparison.loc[smotenc_comparison['strategy'].eq('SMOTENC'),
    'training_rows_after_sampling'].iloc[0]
smote_summary = {
    'fn_reduction_pct': float((base_fn - smote_fn) / base_fn * 100) if base_fn else 0,
    'training_rows_increase_pct': float((smote_rows - base_rows) / base_rows * 100),
    'conclusion': 'SMOTENC is not justified as the default production strategy in this experiment; class weighting
    is simpler.'
}
display(smotenc_comparison)
print(smote_summary)

```

Key Output

strategy	training rows after sampling	precision	recall	f1	pr auc	false negatives	training time sec	threshold
Class weighting	50000	0.0701	0.6673	0.1269	0.0727	1464	3.5099	0.4460
SMOTENC	57038	0.0689	0.6683	0.1250	0.0716	1460	8.8141	0.1440

```
{'fn_reduction_pct': 0.273224043715847, 'training_rows_increase_pct': 14.076, 'conclusion': 'SMOTENC is preferred only if fa
```

Step 7. Business Statistics and User/Profile Analysis

This section computes weekend uplift, product CTR, user profiles and confidence intervals.

Step 7. Business Statistics and User/Profile Analysis - Python Implementation

```

# Business statistics
weekend = train_fe.groupby('is_weekend')['is_click'].agg(['mean', 'sum', 'count']).reset_index()
weekday_ctr = float(weekend.loc[weekend['is_weekend'].eq(0), 'mean'].iloc[0])
weekend_ctr = float(weekend.loc[weekend['is_weekend'].eq(1), 'mean'].iloc[0])
weekend_uplift_pp = (weekend_ctr - weekday_ctr) * 100
weekend_relative_uplift = (weekend_ctr / weekday_ctr - 1) * 100
z_stat, p_value = proportions_ztest(
    weekend.sort_values('is_weekend')['sum'].to_numpy(),
    weekend.sort_values('is_weekend')['count'].to_numpy()
)
print(f'Weekend CTR: {weekend_ctr:.2%}; Weekday CTR: {weekday_ctr:.2%}; uplift: {weekend_uplift_pp:.2f} pp /
    {weekend_relative_uplift:.1f}% relative; p-value={p_value:.4g}')

product = train_fe.groupby('product')['is_click'].agg(clicks='sum', impressions='count', ctr='mean').reset_index()
product['expected_clicks_per_100k_impressions'] = product['ctr'] * 100000
product = product.sort_values('ctr', ascending=False)
display(product)

profile = train_fe.groupby(['gender', 'age_level', 'city_development_index'])['is_click'].agg(clicks='sum',
    impressions='count', ctr='mean').reset_index()
profile = profile[profile['impressions'] >= 500].copy()
ci_low, ci_high = proportion_confint(profile['clicks'], profile['impressions'], method='wilson')
profile['ctr_ci_low'] = ci_low
profile['ctr_ci_high'] = ci_high
profile = profile.sort_values('ctr', ascending=False).head(10)
display(profile)

```

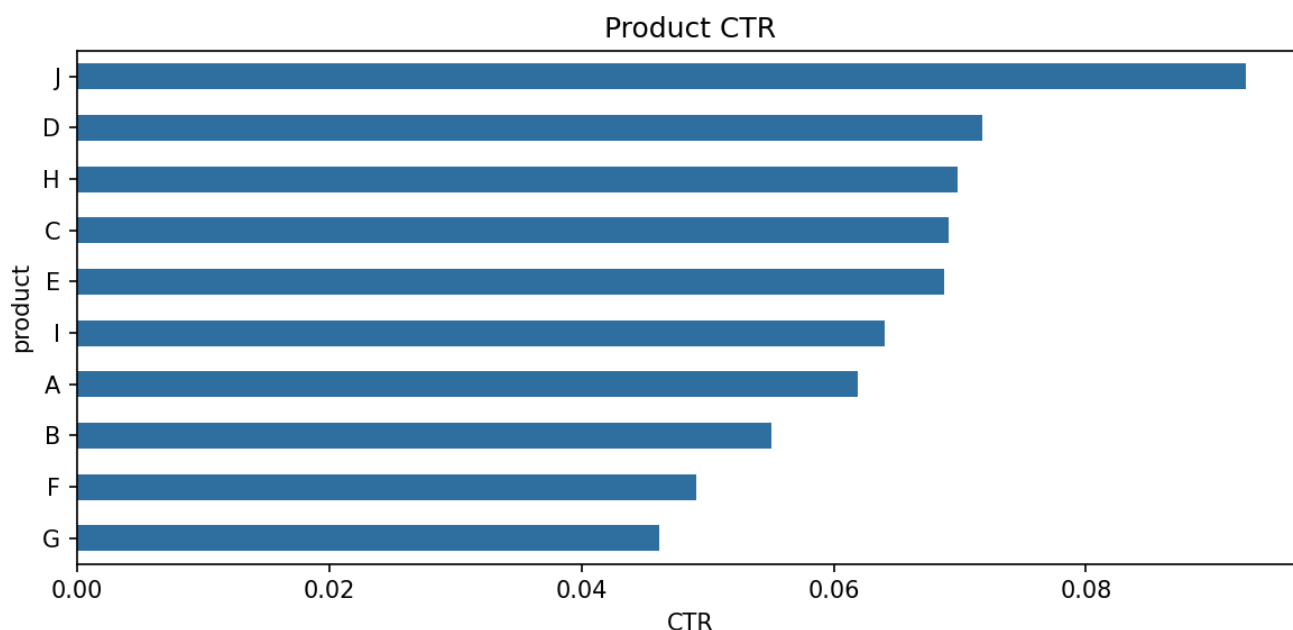
Key Output

```
Weekend CTR: 7.33%; Weekday CTR: 6.65%; uplift: 0.68 pp / 10.2% relative; p-value=4.272e-12
```

product	clicks	impressions	ctr	expected clicks per 100k impressions
J	899	9698	0.0927	9269.95
D	2949	41064	0.0718	7181.47

product	clicks	impressions	ctr	expected clicks per 100k impressions
H	7654	109574	0.0699	6985.23
C	11306	163501	0.0691	6914.94
E	1474	21452	0.0687	6871.15
I	4079	63711	0.0640	6402.35
A	953	15391	0.0619	6191.93
B	1238	22479	0.0551	5507.36
F	344	7007	0.0491	4909.38
G	435	9414	0.0462	4620.78

gender	age level	city development index	clicks	impressions	ctr	ctr ci low	ctr ci high
Male	5.0000	4.0000	255	2945	0.0866	0.0770	0.0973
Female	5.0000	4.0000	100	1181	0.0847	0.0701	0.1019
Male	1.0000	2.0000	605	7922	0.0764	0.0707	0.0824
Female	5.0000	2.0000	208	2752	0.0756	0.0663	0.0861
Male	2.0000	2.0000	3736	50409	0.0741	0.0719	0.0764
Male	1.0000	3.0000	607	8291	0.0732	0.0678	0.0790
Male	1.0000	4.0000	306	4252	0.0720	0.0646	0.0801
Male	5.0000	3.0000	349	4909	0.0711	0.0642	0.0786
Male	2.0000	3.0000	1927	27671	0.0696	0.0667	0.0727
Male	5.0000	2.0000	493	7112	0.0693	0.0636	0.0755



Step 8. Native Feature Importance

Native model importance is extracted for interpretability.

Step 8. Native Feature Importance - Python Implementation

```
# Top individual feature importance; keep exact names instead of broad split('_')[0] grouping.
feature_names = best_pipe.named_steps['prep'].get_feature_names_out()
importances = getattr(best_pipe.named_steps['model'], 'feature_importances_', None)
if importances is not None:
```

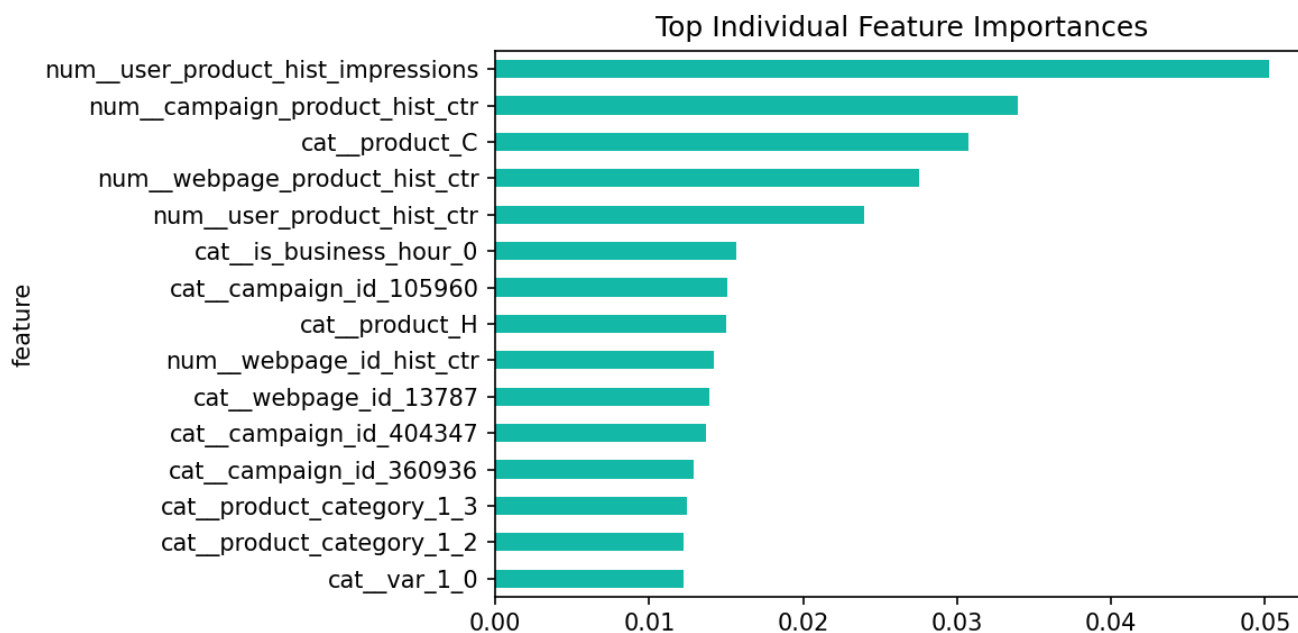
```

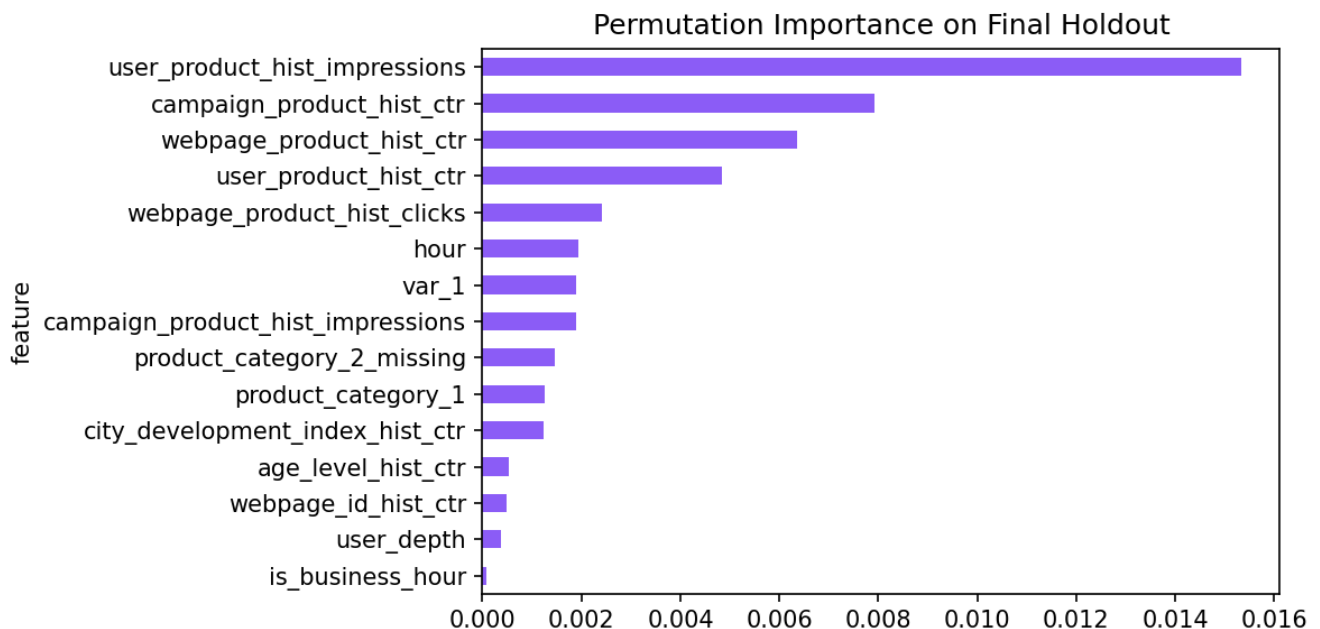
feature_importance = pd.DataFrame({'feature': feature_names, 'importance':
importances}).sort_values('importance', ascending=False).head(15)
else:
    sample_n = min(15000, len(holdout_hist))
    sample = holdout_hist.sample(sample_n, random_state=RANDOM_STATE)
    perm = permutation_importance(best_pipe, sample[full_features], sample['is_click'],
scoring='average_precision', n_repeats=3, random_state=RANDOM_STATE, n_jobs=-1)
    feature_importance = pd.DataFrame({'feature': full_features, 'importance':
perm.importances_mean}).sort_values('importance', ascending=False).head(15)
display(feature_importance)

```

Key Output

feature	importance
num__user_product_hist_impressions	0.0503
num__campaign_product_hist_ctr	0.0339
cat__product_C	0.0307
num__webpage_product_hist_ctr	0.0275
num__user_product_hist_ctr	0.0239
cat__is_business_hour_0	0.0156
cat__campaign_id_105960	0.0150
cat__product_H	0.0150
num__webpage_id_hist_ctr	0.0142
cat__webpage_id_13787	0.0139





Step 9. ROC, Precision-Recall, and Calibration Charts

Evaluation curves are generated from final holdout predictions.

Step 9. ROC, Precision-Recall, and Calibration Charts - Python Implementation

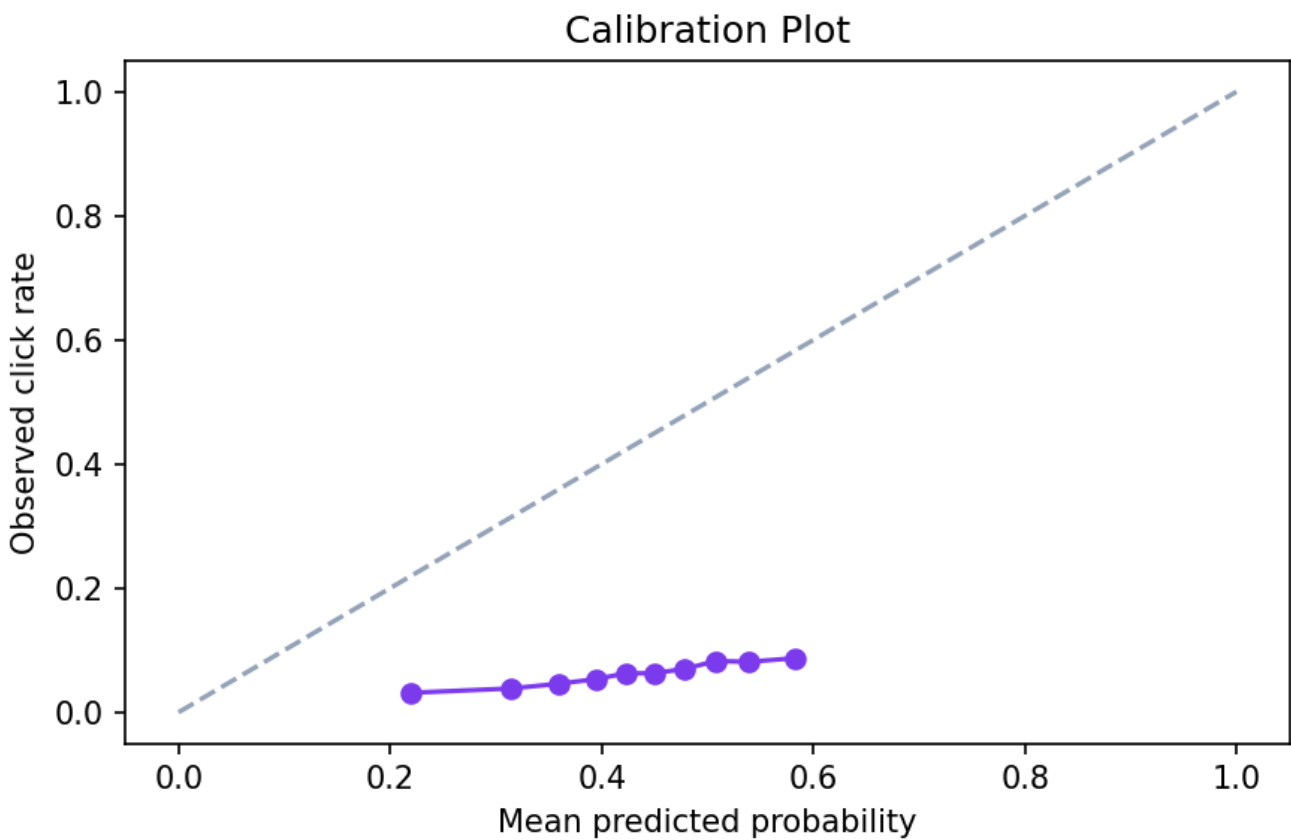
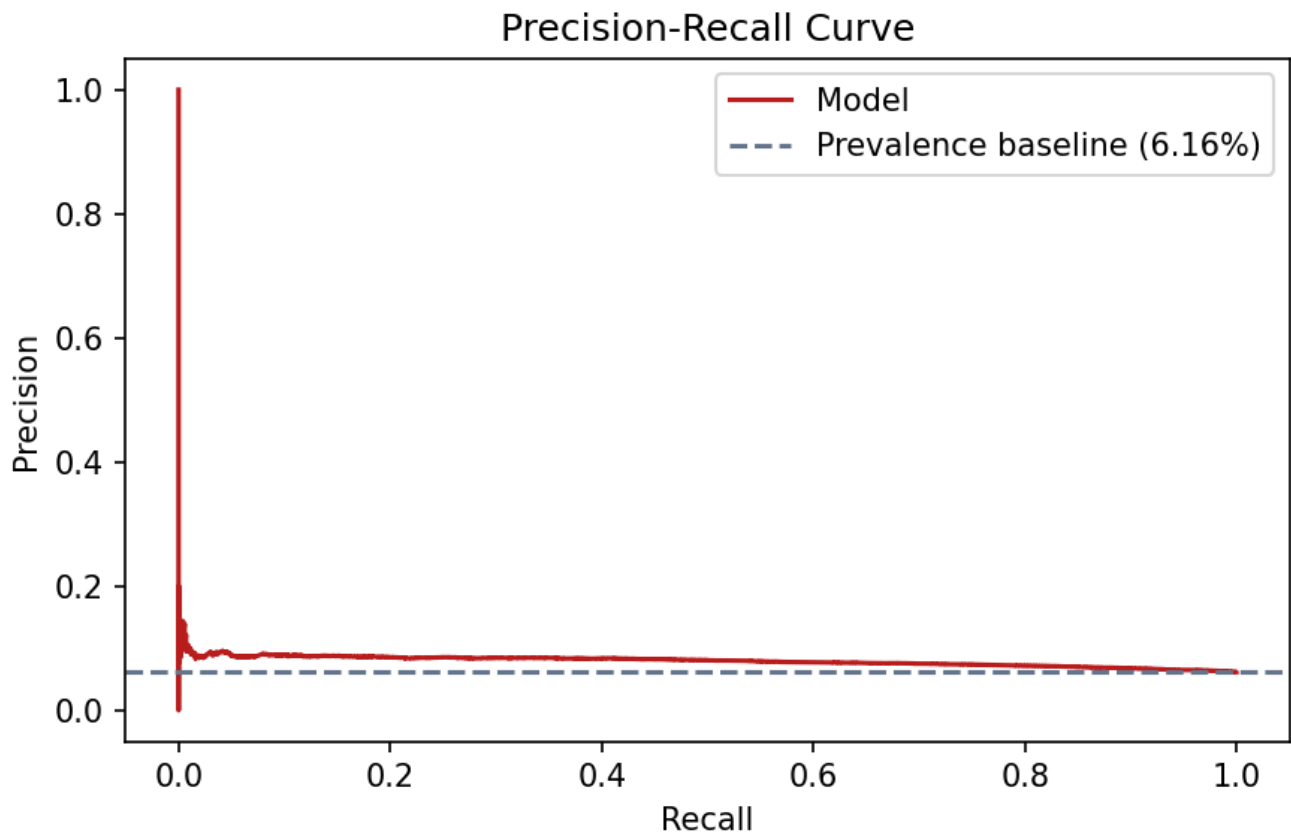
```
# Curves and charts
plt.figure(figsize=(6,4)); train['is_click'].value_counts().sort_index().plot(kind='bar',
    color=['#64748b', '#f59e0b']); plt.xticks([0,1], ['No Click', 'Click'], rotation=0); plt.title('Target
    Distribution'); plt.ylabel('Impressions'); plt.tight_layout(); plt.savefig(IMAGE_DIR/'target_distribution.png',
    dpi=150); plt.close(); display(Image(filename=str(IMAGE_DIR/'target_distribution.png'))))

plt.figure(figsize=(8,4)); product.sort_values('ctr').plot(kind='barh', x='product', y='ctr', legend=False,
    ax=plt.gca(), color='#2f6f9f'); plt.title('Product CTR'); plt.xlabel('CTR'); plt.tight_layout();
    plt.savefig(IMAGE_DIR/'product_ctr.png', dpi=150); plt.close();
    display(Image(filename=str(IMAGE_DIR/'product_ctr.png'))))

plot_metrics = metrics[metrics['model'].str.endswith('tuned threshold')].sort_values('f1')
plt.figure(figsize=(8,4)); plot_metrics.plot(kind='barh', x='model', y='f1', legend=False, ax=plt.gca(),
    color='#f59e0b'); plt.title('Tuned-threshold F1 by Model'); plt.xlabel('F1'); plt.tight_layout();
    plt.savefig(IMAGE_DIR/'model_f1.png', dpi=150); plt.close();
    display(Image(filename=str(IMAGE_DIR/'model_f1.png'))))

plt.figure(figsize=(8,4)); ablation.sort_values('f1').plot(kind='barh', x='model', y='f1', legend=False,
    ax=plt.gca(), color='#0f766e'); plt.title('Feature Ablation: F1 Lift'); plt.xlabel('F1'); plt.tight_layout();
    plt.savefig(IMAGE_DIR/'feature_ablation.png', dpi=150); plt.close();
    display(Image(filename=str(IMAGE_DIR/'feature_ablation.png'))))

plt.figure(figsize=(8,4)); feature_importance.sort_values('importance').plot(kind='barh', x='feature',
    y='importance', legend=False, ax=plt.gca(), color='#14b8a6'); plt.title('Top Individual Feature Importances');
    plt.tight_layout(); plt.savefig(IMAGE_DIR/'feature_importance.png', dpi=150); plt.close();
    display(Image(filename=str(IMAGE_DIR/'feature_importance.png'))))
```



Step 10. Permutation Importance on Final Holdout

Permutation importance is computed for interpretation only, not tuning.

Step 10. Permutation Importance on Final Holdout - Python Implementation

```
# Model-agnostic interpretability on untouched holdout. Interpretation only; no tuning.
perm_sample_n = min(3000, len(holdout_hist))
perm_sample = holdout_hist.sample(perm_sample_n, random_state=RANDOM_STATE)
perm_result = permutation_importance(
    best_pipe,
    perm_sample[full_features],
    perm_sample['is_click'],
    scoring='average_precision',
    n_repeats=2,
    random_state=RANDOM_STATE,
    n_jobs=-1,
)
permutation_importance_table = (
    pd.DataFrame({'feature': full_features, 'importance': perm_result.importances_mean})
    .sort_values('importance', ascending=False)
    .head(15)
)
display(permutation_importance_table)
plt.figure(figsize=(8,4)); permutation_importance_table.sort_values('importance').plot(kind='barh', x='feature',
    y='importance', legend=False, ax=plt.gca(), color='#8b5cf6'); plt.title('Permutation Importance on Final
    Holdout'); plt.tight_layout(); plt.savefig(IMAGE_DIR/'permutation_importance.png', dpi=150); plt.close();
    display(Image(filename=str(IMAGE_DIR/'permutation_importance.png'))))
display(Markdown("""**Interpretability note.** Native selected-model importance shows how the fitted model used
    features during training. Permutation importance estimates how much holdout PR-AUC degrades when each feature
    is disrupted, making it model-agnostic and closer to final-holdout behavior. Differences between the two views
    should be treated as complementary, not contradictory."""))

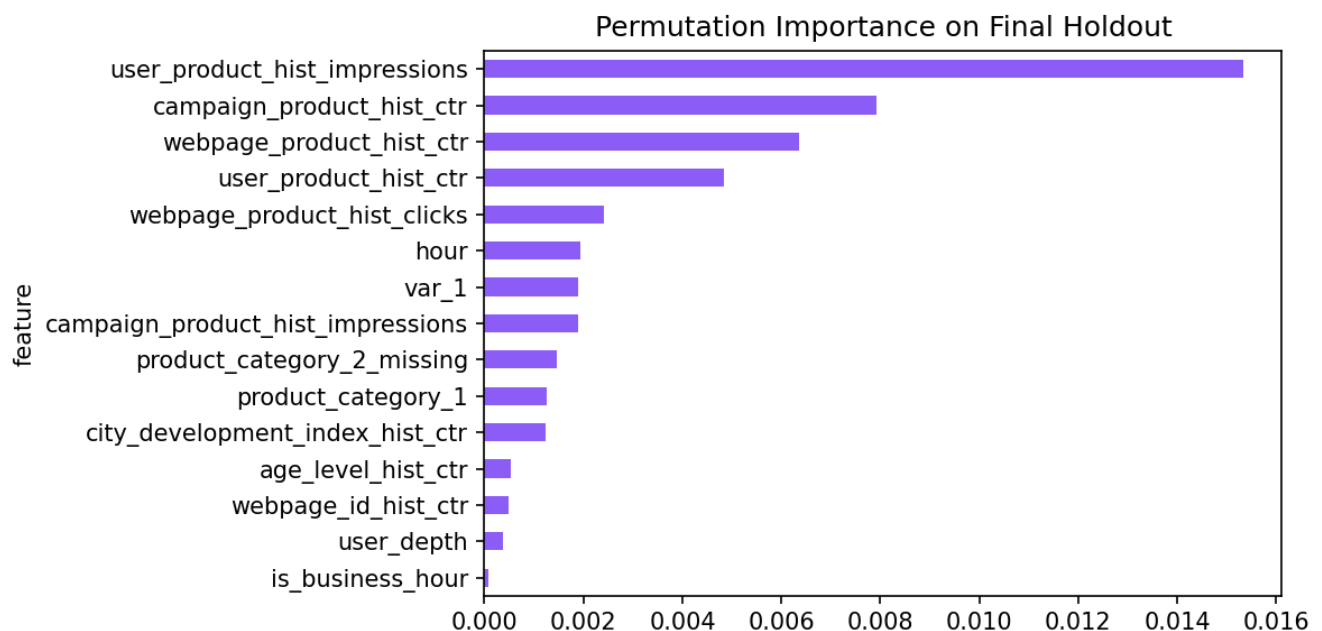
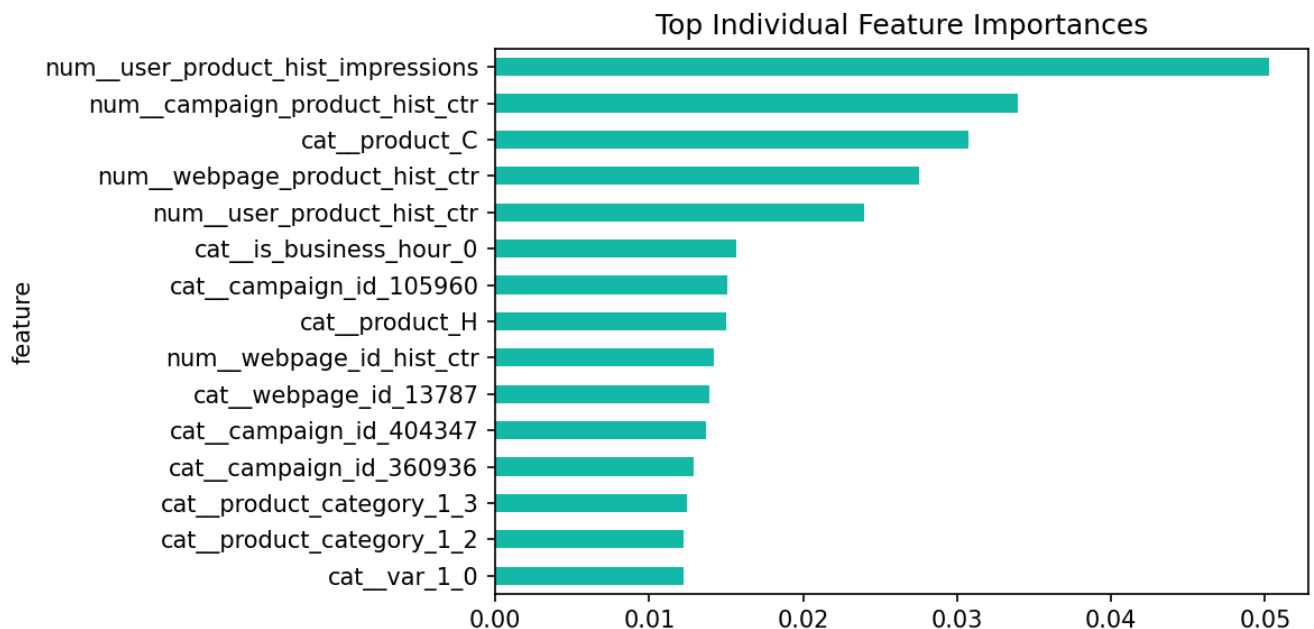
precision, recall, _ = precision_recall_curve(y_valid, best_probs)
plt.figure(figsize=(6,4)); plt.plot(recall, precision, color='#b91c1c', label='Model'); plt.axhline(y_valid.mean(),
    color='#64748b', linestyle='--', label=f'Prevalence baseline ({y_valid.mean():.2%})'); plt.title('Precision-
    Recall Curve'); plt.xlabel('Recall'); plt.ylabel('Precision'); plt.legend(); plt.tight_layout();
    plt.savefig(IMAGE_DIR/'pr_curve.png', dpi=150); plt.close();
    display(Image(filename=str(IMAGE_DIR/'pr_curve.png'))))

fpr, tpr, _ = roc_curve(y_valid, best_probs)
plt.figure(figsize=(6,4)); plt.plot(fpr, tpr, color='#2563eb'); plt.plot([0,1],[0,1], '--', color='#94a3b8');
    plt.title('ROC Curve'); plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate');
    plt.tight_layout(); plt.savefig(IMAGE_DIR/'roc_curve.png', dpi=150); plt.close();
    display(Image(filename=str(IMAGE_DIR/'roc_curve.png'))))

prob_true, prob_pred = calibration_curve(y_valid, best_probs, n_bins=10, strategy='quantile')
plt.figure(figsize=(6,4)); plt.plot(prob_pred, prob_true, marker='o', color='#7c3aed');
    plt.plot([0,1],[0,1], '--', color='#94a3b8'); plt.title('Calibration Plot'); plt.xlabel('Mean predicted
    probability'); plt.ylabel('Observed click rate'); plt.tight_layout();
    plt.savefig(IMAGE_DIR/'calibration_plot.png', dpi=150); plt.close();
    display(Image(filename=str(IMAGE_DIR/'calibration_plot.png'))))
```

Key Output

feature	importance
user_product_hist_impressions	0.0153
campaign_product_hist_ctr	0.0079
webpage_product_hist_ctr	0.0064
user_product_hist_ctr	0.0048
webpage_product_hist_clicks	0.0024
hour	0.0019
var_1	0.0019
campaign_product_hist_impressions	0.0019
product_category_2_missing	0.0015
product_category_1	0.0013



Step 11. Lift, Precision@K, Business Value, Bootstrap, and Data Quality

Ranking, business-value sensitivity, bootstrap uncertainty, data quality and cold-start analyses are calculated.

Step 11. Lift, Precision@K, Business Value, Bootstrap, and Data Quality - Python Implementation

```
# Decision-oriented evaluation: lift/gains, Precision@K/Recall@K, cost thresholds, CIs, data quality, and cold
start.

def make_lift_table(y_true, scores, n_bins=10):
    frame = pd.DataFrame({'actual': np.asarray(y_true), 'score': np.asarray(scores)})
    frame = frame.sort_values('score', ascending=False).reset_index(drop=True)
    frame['decile'] = pd.qcut(frame.index + 1, q=n_bins, labels=[f'Top {i*10}-{(i+1)*10}%' for i in range(n_bins)],
                             duplicates='drop')
    baseline_ctr = frame['actual'].mean()
    lift = frame.groupby('decile', observed=True).agg(
```

```

        impressions=('actual','size'), clicks=('actual','sum'), actual_ctr=('actual','mean'),
        avg_score=('score','mean')
    ).reset_index()
    lift['lift_vs_baseline'] = lift['actual_ctr'] / baseline_ctr
    lift['clicks_captured_pct'] = lift['clicks'] / frame['actual'].sum()
    lift['cumulative_clicks_captured_pct'] = lift['clicks'].cumsum() / frame['actual'].sum()
    return lift

def precision_recall_at_k(y_true, scores, ks=(0.01, 0.05, 0.10, 0.20)):
    frame = pd.DataFrame({'actual': np.asarray(y_true), 'score': np.asarray(scores)}).sort_values('score',
    ascending=False)
    total_clicks = frame['actual'].sum()
    baseline = frame['actual'].mean()
    rows = []
    for k in ks:
        top = frame.head(max(1, int(len(frame) * k)))
        precision_k = top['actual'].mean()
        rows.append({
            'targeted_impressions': f'Top {int(k*100)}%',
            'impressions_served': len(top),
            'precision_at_k': precision_k,
            'recall_at_k': top['actual'].sum() / total_clicks,
            'lift': precision_k / baseline,
        })
    return pd.DataFrame(rows)

def business_value_threshold_table(tune_y, tune_scores, holdout_y, holdout_scores, scenarios):
    """Select business thresholds on tuning data, then evaluate them on final holdout.
    Values are assumptions because the dataset has no CPC, CPM, margin, or revenue fields.
    Utility = true_clicks_served * value_per_click - served_impressions * cost_per_impression.
    """
    thresholds = np.unique(np.round(np.linspace(0.01, 0.99, 199), 3))
    tune_y = np.asarray(tune_y)
    holdout_y = np.asarray(holdout_y)
    total_holdout_clicks = holdout_y.sum()
    rows = []
    for scenario, value_per_click, cost_per_impression in scenarios:
        best_tuning = None
        for threshold in thresholds:
            tune_pred = (tune_scores >= threshold).astype(int)
            tn, fp, fn, tp = confusion_matrix(tune_y, tune_pred).ravel()
            served = tp + fp
            utility = tp * value_per_click - served * cost_per_impression
            if best_tuning is None or utility > best_tuning['tuning_expected_utility']:
                best_tuning = {'optimal_threshold': threshold, 'tuning_expected_utility': utility}
        threshold = best_tuning['optimal_threshold']
        holdout_pred = (holdout_scores >= threshold).astype(int)
        tn, fp, fn, tp = confusion_matrix(holdout_y, holdout_pred).ravel()
        served = tp + fp
        holdout_utility = tp * value_per_click - served * cost_per_impression
        rows.append({
            'scenario': scenario,
            'value_per_click': value_per_click,
            'cost_per_impression': cost_per_impression,
            'selected_on': 'threshold_tuning_split',
            'evaluated_on': 'final_holdout',
            'optimal_threshold': threshold,
            'served_impressions': int(served),
            'clicks_captured': int(tp),
            'clicks_missed': int(fn),
            'precision': precision_score(holdout_y, holdout_pred, zero_division=0),
            'recall': recall_score(holdout_y, holdout_pred, zero_division=0),
            'tuning_expected_utility': best_tuning['tuning_expected_utility'],
            'holdout_expected_utility': holdout_utility,
            'holdout_utility_per_1k_impressions': holdout_utility / len(holdout_y) * 1000,
            'clicks_captured_pct': tp / total_holdout_clicks if total_holdout_clicks else 0,
        })
    return pd.DataFrame(rows)

def bootstrap_metric_ci(y_true, model_score_pairs, n_boot=200):
    rng = np.random.default_rng(RANDOM_STATE)
    y_arr = np.asarray(y_true)
    rows = []
    for name, scores, threshold in model_score_pairs:
        scores = np.asarray(scores)
        f1_values, pr_values = [], []
        for _ in range(n_boot):
            idx = rng.integers(0, len(y_arr), len(y_arr))
            if len(np.unique(y_arr[idx])) < 2:
                continue
            f1_values.append(f1_score(y_arr[idx], scores[idx] >= threshold, zero_division=0))
            pr_values.append(average_precision_score(y_arr[idx], scores[idx]))
        rows.append({
            'model': name,
            'f1': f1_score(y_arr, scores >= threshold, zero_division=0),
            'f1_ci_low': np.percentile(f1_values, 2.5),
            'f1_ci_high': np.percentile(f1_values, 97.5),
            'pr_auc': average_precision_score(y_arr, scores),
            'pr_auc_ci_low': np.percentile(pr_values, 2.5),
            'pr_auc_ci_high': np.percentile(pr_values, 97.5),
        })

```

```

    })
    return pd.DataFrame(rows)

def cold_start_table(frame, scores, y_true):
    out = frame.copy()
    out['score'] = np.asarray(scores)
    out['actual'] = np.asarray(y_true)
    rows = []
    for feature in ['user_product', 'campaign_product', 'webpage_product']:
        known = out[f'{feature}_hist_impressions'] > 0
        for label, mask in [(('known_history', known), ('cold_start', ~known))]:
            subset = out[mask]
            rows.append({
                'feature': feature,
                'segment': label,
                'impressions': len(subset),
                'share_of_holdout': len(subset) / len(out),
                'ctr': subset['actual'].mean(),
                'roc_auc': roc_auc_score(subset['actual'], subset['score']) if subset['actual'].nunique() == 2 else
                np.nan,
                'pr_auc': average_precision_score(subset['actual'], subset['score']) if subset['actual'].nunique()
                == 2 else np.nan,
            })
    return pd.DataFrame(rows)

data_quality = pd.DataFrame([
    ['Training observations', len(train)],
    ['Test observations', len(test)],
    ['Clicks', int(train['is_click'].sum())],
    ['Non-clicks', int((train['is_click'] == 0).sum())],
    ['Click rate', f"{train['is_click'].mean():.2%}"],
    ['Duplicate rows', int(train.duplicated().sum())],
    ['Total missing cells', int(train.isna().sum().sum())],
    ['Unique products', train['product'].nunique()],
    ['Unique campaigns', train['campaign_id'].nunique()],
    ['Unique webpages', train['webpage_id'].nunique()],
    ['Date range', f"{train['DateTime'].min()} to {train['DateTime'].max()}"],
], columns=['Data check', 'Result'])
missing_summary =
    (train.isna().mean().mul(100).sort_values(ascending=False).reset_index().rename(columns={'index': 'field',
    0: 'missing_pct'})).query('missing_pct > 0')

decile_lift = make_lift_table(y_valid, best_probs)
precision_k = precision_recall_at_k(y_valid, best_probs)
business_value_scenarios = business_value_threshold_table(threshold_hist['is_click'], best_threshold_probs,
    y_valid, best_probs, [
    ('Low click value / cheap media', 1.00, 0.05),
    ('Base ad economics', 2.00, 0.05),
    ('High click value', 5.00, 0.05),
    ('Expensive media', 2.00, 0.15),
])
model_score_pairs = []
for model_name, scores in probs_by_model.items():
    threshold = float(metrics.loc[metrics['model'].eq(model_name + ' - tuned threshold'), 'threshold'].iloc[0])
    model_score_pairs.append((model_name, scores, threshold))
bootstrap_ci = bootstrap_metric_ci(y_valid, model_score_pairs)
cold_start = cold_start_table(holdout_hist, best_probs, y_valid)

display(data_quality)
display(missing_summary)

display(Markdown(f"****Temporal CTR note.** Overall training CTR is `{train['is_click'].mean():.2%}`, while the
    untouched final holdout CTR is `{holdout_hist['is_click'].mean():.2%}`. This temporal variation reinforces
    chronological validation and ongoing CTR/feature-drift monitoring after deployment.

**Missing-value handling.** The pipeline uses most-frequent imputation for categorical/context fields and median
    imputation for numeric historical features inside sklearn pipelines. Because `product_category_2` has very high
    missingness, the notebook adds `product_category_2_missing` consistently across train, tuning, holdout, and
    test data. High missingness may itself contain predictive signal, but this is associative rather than
    causal.""))
display(decile_lift)
display(precision_k)
display(Markdown(f"****Ranking interpretation.** Although overall classification metrics are moderate, the model
    demonstrates useful ranking value. On the untouched final holdout, the top 10% of ranked impressions achieve
    `{decile_lift.iloc[0]['actual_ctr']:.2%}` CTR versus the `{y_valid.mean():.2%}` holdout baseline, producing
    `{decile_lift.iloc[0]['lift_vs_baseline']:.2f}x` lift and capturing
    `{decile_lift.iloc[0]['clicks_captured_pct']:.1%}` of all clicks. The top 20% captures approximately
    `{decile_lift.iloc[:2]['clicks'].sum() / decile_lift['clicks'].sum():.1%}` of clicks. Top 1% lift is
    `{precision_k.loc[precision_k['targeted_impressions'].eq('Top 1%'), 'lift'].iloc[0]:.2f}x`. Therefore, the
    model is better suited to budget-constrained ad ranking than hard click/no-click classification.""))
display(business_value_scenarios)
display(bootstrap_ci)
display(Markdown(f"****Model-selection interpretation.** XGBoost is selected under the predefined F1 decision rule.
    However, bootstrap confidence intervals overlap with Logistic Regression, so the results do not provide strong
    evidence that XGBoost is statistically superior. Logistic Regression remains a reasonable alternative when
    interpretability and serving simplicity are prioritized.""))
display(cold_start)
user_cold = cold_start[(cold_start['feature'].eq('user_product')) &
    (cold_start['segment'].eq('cold_start'))].iloc[0]
user_known = cold_start[(cold_start['feature'].eq('user_product')) &
    (cold_start['segment'].eq('known_history'))].iloc[0]

```

```
campaign_cold = cold_start[(cold_start['feature'].eq('campaign_product')) &
(cold_start['segment'].eq('cold_start'))].iloc[0]
webpage_cold = cold_start[(cold_start['feature'].eq('webpage_product')) &
(cold_start['segment'].eq('cold_start'))].iloc[0]
display(Markdown(f""**Cold-start interpretation.** User-product cold-start impressions represent
`{user_cold['share_of_holdout']:.1%}` of the final holdout, while campaign-product and webpage-product cold-
start rates are `{campaign_cold['share_of_holdout']:.1%}` and `{webpage_cold['share_of_holdout']:.1%}`.
Production scoring therefore cannot depend primarily on user-product affinity; campaign, webpage, product,
demographic, temporal, and contextual signals should provide fallback ranking signal. Cold-start user-product
CTR (`{user_cold['ctr']:.2%}`) is higher than known-history CTR (`{user_known['ctr']:.2%}`), but this should
not be interpreted as proof that cold-start users are intrinsically better; it may reflect segment/composition
differences and should be monitored."""))
```

Key Output

Data check	Result
Training observations	463291
Test observations	128858
Clicks	31331
Non-clicks	431960
Click rate	6.76%
Duplicate rows	0
Total missing cells	563955
Unique products	10
Unique campaigns	10
Unique webpages	9

field	missing pct
product_category_2	78.97
city_development_index	27.01
gender	3.9377
user_group_id	3.9377
age_level	3.9377
user_depth	3.9377

decile	impressions	clicks	actual ctr	avg score	lift vs baseline	clicks captured pct	cumulative clicks captured pct
Top 0-10%	7147	624	0.0873	0.5826	1.4178	0.1418	0.1418
Top 10-20%	7147	580	0.0812	0.5388	1.3178	0.1318	0.2736
Top 20-30%	7146	591	0.0827	0.5081	1.3430	0.1343	0.4079
Top 30-40%	7147	499	0.0698	0.4779	1.1338	0.1134	0.5212
Top 40-50%	7146	449	0.0628	0.4492	1.0203	0.1020	0.6233
Top 50-60%	7147	449	0.0628	0.4227	1.0202	0.1020	0.7253
Top 60-70%	7146	383	0.0536	0.3952	0.8703	0.0870	0.8123
Top 70-80%	7147	331	0.0463	0.3587	0.7521	0.0752	0.8875
Top 80-90%	7146	271	0.0379	0.3142	0.6158	0.0616	0.9491
Top 90-100%	7147	224	0.0313	0.2187	0.5089	0.0509	1.0000

targeted impressions	impressions served	precision at k	recall at k	lift
Top 1%	714	0.0910	0.0148	1.4783
Top 5%	3573	0.0873	0.0709	1.4180
Top 10%	7146	0.0873	0.1418	1.4180

targeted impressions	impressions served	precision at k	recall at k	lift
Top 20%	14293	0.0842	0.2736	1.3679

scenario	value per click	cost per impression	selected on	evaluated on	optimal threshold	served impressions	clicks
Low click value / cheap media	1.0000	0.0500	threshold_tuning_split	final_holdout	0.3910	47296	3429
Base ad economics	2.0000	0.0500	threshold_tuning_split	final_holdout	0.1580	70411	4389
High click value	5.0000	0.0500	threshold_tuning_split	final_holdout	0.1140	70943	4399
Expensive media	2.0000	0.1500	threshold_tuning_split	final_holdout	0.5200	15115	1279

model	f1	f1 ci low	f1 ci high	pr auc	pr auc ci low	pr auc ci high
Logistic Regression - balanced	0.1371	0.1321	0.1420	0.0815	0.0782	0.0850
Random Forest - balanced	0.1292	0.1223	0.1369	0.0790	0.0755	0.0827
XGBoost - weighted	0.1376	0.1323	0.1435	0.0795	0.0761	0.0829

feature	segment	impressions	share of holdout	ctr	roc auc	pr auc
user_product	known_history	35923	0.5027	0.0495	0.5892	0.0662
user_product	cold_start	35543	0.4973	0.0738	0.5550	0.0849
campaign_product	known_history	71114	0.9951	0.0615	0.5909	0.0795
campaign_product	cold_start	352	0.0049	0.0682	0.4999	0.1071
webpage_product	known_history	71114	0.9951	0.0615	0.5909	0.0795
webpage_product	cold_start	352	0.0049	0.0682	0.4999	0.1071

Step 12. Production Architecture / ML Lifecycle

A readable lifecycle diagram links scoring, monitoring and retraining.

Step 12. Production Architecture / ML Lifecycle - Python Implementation

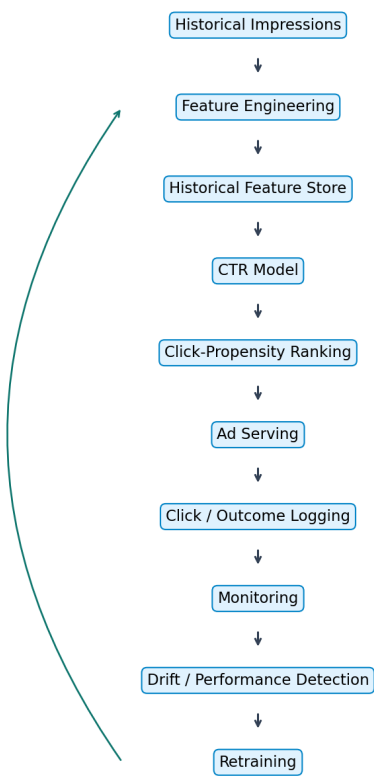
```
# Production lifecycle diagram.
fig, ax = plt.subplots(figsize=(7.5, 9.5))
ax.axis('off')
steps = [
    'Historical Impressions', 'Feature Engineering', 'Historical Feature Store',
    'CTR Model', 'Click-Propensity Ranking', 'Ad Serving', 'Click / Outcome Logging',
    'Monitoring', 'Drift / Performance Detection', 'Retraining'
]
y_positions = np.linspace(0.93, 0.10, len(steps))
x = 0.5
for i, (y, step) in enumerate(zip(y_positions, steps)):
    ax.text(x, y, step, ha='center', va='center', fontsize=11, bbox=dict(boxstyle='round,pad=0.35',
        facecolor='#e0f2fe', edgecolor='#0284c7'))
    if i < len(steps) - 1:
        ax.annotate('', xy=(x, y_positions[i+1] + 0.035), xytext=(x, y - 0.035), arrowprops=dict(arrowstyle='->',
            color='#334155', lw=1.5))
ax.annotate('', xy=(0.31, y_positions[1]), xytext=(0.31, y_positions[-1]), arrowprops=dict(arrowstyle='->',
    color='#0f766e', lw=1.3, connectionstyle='arc3,rad=-0.35'))
ax.text(0.5, 0.03, 'Monitor: PR-AUC | Lift@K | CTR | Calibration | Feature Drift | Latency', ha='center',
    fontsize=10, color='#475569')
ax.set_title('Production Lifecycle for CTR Scoring', fontsize=15, fontweight='bold')
plt.tight_layout(); plt.savefig(IMAGE_DIR/'production_lifecycle.png', dpi=150); plt.close();
display(Image(filename=str(IMAGE_DIR/'production_lifecycle.png')))
```

```
business_mapping = pd.DataFrame([
    ['High product CTR', 'Increase exposure and reserve more inventory'],
    ['High webpage CTR', 'Prioritize placement and monitor page quality'],
    ['High user-product historical CTR', 'Personalize ad selection and test targeted bid uplift'],
    ['Weekend uplift', 'Test weekend bid modifier before permanent adoption'],
    ['High predicted click propensity', 'Prioritize impression under budget constraints'],
    ['Low CTR product', 'Refresh creative or reduce allocation'],
    ['Model drift', 'Retrain and recalibrate thresholds'],
], columns=['Model insight', 'Business action'])
display(business_mapping)
```

Key Output

Model insight	Business action
High product CTR	Increase exposure and reserve more inventory
High webpage CTR	Prioritize placement and monitor page quality
High user-product historical CTR	Personalize ad selection and test targeted bid...
Weekend uplift	Test weekend bid modifier before permanent ado...
High predicted click propensity	Prioritize impression under budget constraints
Low CTR product	Refresh creative or reduce allocation
Model drift	Retrain and recalibrate thresholds

Production Lifecycle for CTR Scoring



Monitor: PR-AUC | Lift@K | CTR | Calibration | Feature Drift | Latency

Step 13. Final Retraining and Test Prediction Generation

The selected architecture is retrained on eligible labeled history and used to score the test data.

Step 13. Final Retraining and Test Prediction Generation - Python Implementation

```
# Final train/test feature construction: training rows are cumulative; test sees all train history.
full_train_hist, full_prior = add_cumulative_history_features(train_fe, history_cols)
full_maps, _ = build_history_maps(train_fe, history_cols)
test_final = apply_history_maps(test_fe, full_maps, full_prior, history_cols)
final_pipe = Pipeline([['prep', make_preprocess(all_numeric_features)), ('model', models[best_name])])
# Final fit freezes the selected architecture and threshold, then uses all eligible
# historical training rows before scoring the test data. This keeps the threshold
# selection independent while using maximum available history for final scoring.
```

```

final_pipe.fit(full_train_hist[full_features], full_train_hist['is_click'])
test_pred = final_pipe.predict_proba(test_final[full_features])[:, 1]
submission = pd.DataFrame({
    'session_id': test['session_id'],
    'click_propensity_score': test_pred,
    'predicted_click': (test_pred >= best_threshold).astype(int),
})
submission.to_csv(OUTPUT_DIR/'ad_click_prediction_test_predictions.csv', index=False)
print(submission.head())

artifacts = {
    'train_shape': train.shape,
    'test_shape': test.shape,
    'click_rate': float(train['is_click'].mean()),
    'training_period_end_before': str(threshold_cutoff.date()),
    'threshold_tuning_date': str(threshold_cutoff.date()),
    'final_holdout_date': str(holdout_cutoff.date()),
    'metrics': metrics.to_dict('records'),
    'ablation': ablation.to_dict('records'),
    'best_model': best_name,
    'best_threshold': best_threshold,
    'weekend': weekend.to_dict('records'),
    'weekend_uplift_pp': weekend_uplift_pp,
    'weekend_relative_uplift_pct': weekend_relative_uplift,
    'weekend_p_value': float(p_value),
    'product': product.to_dict('records'),
    'profile': profile.to_dict('records'),
    'feature_importance': feature_importance.to_dict('records'),
    'permutation_importance': permutation_importance_table.to_dict('records'),
    'business_mapping': business_mapping.to_dict('records'),
    'data_quality': data_quality.to_dict('records'),
    'missing_summary': missing_summary.to_dict('records'),
    'decile_lift': decile_lift.to_dict('records'),
    'precision_at_k': precision_k.to_dict('records'),
    'business_value_thresholds': business_value_scenarios.to_dict('records'),
    'smotenc_comparison': smotenc_comparison.to_dict('records'),
    'smote_summary': smote_summary,
    'bootstrap_ci': bootstrap_ci.to_dict('records'),
    'cold_start': cold_start.to_dict('records'),
    'performance_note': 'Moderate discrimination and imperfect calibration; use scores as click-propensity rankings until calibrated.',
    'final_training_note': 'The selected architecture and threshold are frozen after temporal validation; the final scoring model is retrained on all available labeled history before predicting the test set.'
}
(OUTPUT_DIR/'ctr_artifacts.json').write_text(json.dumps(artifacts, indent=2, default=str))
print('Artifacts written to outputs/ and assets/images/.')

```

Key Output

	session_id	click_propensity_score	predicted_click
0	411705	0.594859	1
1	208263	0.242020	0
2	239450	0.262279	0
3	547761	0.331186	0
4	574275	0.555911	1

Artifacts written to outputs/ and assets/images/.

Step 14. Business Questions 1-7: Explicit Answer Map

1. Weekend vs weekday: Weekend CTR is computed with a two-proportion test; the observed uplift is treated as a bid-test hypothesis, not causal proof.
2. Product CTR: Product-level CTR and expected clicks per 100K impressions identify high- and low-performing products for inventory planning and creative review.
3. Personalized interactions: Feature ablation shows interaction history and leakage-safe historical CTR features improve ranking metrics over raw/contextual features.
4. Main click drivers: Native selected-model importance and model-agnostic permutation importance are used together; historical affinity and contextual features are interpreted as associative drivers.
5. SMOTENC / imbalance: In this experiment, SMOTENC is not justified as the default production strategy: it reduces false negatives by only 0.27% while increasing training rows by 14.08%, increasing training time, and slightly reducing

PR-AUC. Class weighting is therefore the simpler default.

6. Inventory planning: Product CTR is converted into expected clicks per 100K impressions as a planning signal, not a formal demand forecast.

7. User profiles: High-propensity profile segments use minimum-volume filtering and Wilson intervals; recommendations require fairness, privacy, drift, and A/B-test validation.

Step 15. Limitations and Future Improvements

- Validation metrics show moderate discrimination, not high accuracy.
- Add more behavioral history, creative metadata, bid price, placement quality, and recency features if available.
- Use SMOTENC only with a categorical-aware implementation before one-hot encoding.
- Add probability calibration and live A/B testing before production deployment.
- Monitor drift, segment stability, fairness/privacy constraints, and threshold performance over time.

Final Conclusion

The notebook demonstrates a reproducible CTR pipeline with visible implementation code. Outputs should be used as click-propensity ranking signals, monitored for drift/calibration, and validated with A/B testing before causal business rollout.